GENERATIVE AI FOR SOFTWARE ENGINEERS · LECTURE 2 OF 2

From prompts to programs

Generative programming in Java with LangChain4j and Ollama, through a real-estate case study

Felipe Rivera, PhD · University of Victoria

SENG265 · November 5 2026

RECAP

From prompts to programs

The problem with prompting

01 **Silent failures:** nothing tells you an output is wrong.

02 **Untestable outputs:** free-form text resists unit tests.

03 **Brittle behaviour:** small wording changes, different results.

04 **No guarantees:** downstream code must guess the shape.

Generative computing

Imperative
The program is the logic

→

Inductive
Behaviour learned from data

→

Generative
LLM calls woven into code

A **generative function** is a Java method whose body is a call to the model. Types, checks and repairs are ordinary Java code that you can test.

> IBM's Mellea library did this in Python. Today: the same ideas in Java.

PLAN

Four steps, one running example

Running example: our real-estate platform. Owners’ notes in; checked listings, fair rents and ready-to-post ads out.

Step 1 · 10 min

Setup and first call

Ollama, Maven, ChatModel

Step 2 · 10 min

Typed answers

record, enum, interface

Step 3 · 10 min

Check and repair

plain Java checks, a retry loop

Step 4 · 5 min

Majority vote

ask 5 times, count

Each step is one Java class in the companion project genai-java-lab: realestate.Step1FirstCall to realestate.Step4MajorityVote.

THE TOOLS

Three tools, one laptop

Maven

Downloads the libraries your code needs, then compiles and runs it.

Your Java program

One class per step. It calls the model through the LangChain4j library.

Ollama

Runs the model granite4.2:3b on your laptop and answers on localhost:11434.

Why LangChain4j?

A plain Java library for calling language models: typed answers, tools and memory, with no framework to learn. We use version 1.23.0 (October 2026, Apache 2.0). Other Java options: Spring AI for Spring Boot apps, Embabel, and Koog for Kotlin.

Everything runs offline: no API keys, no cloud, no bills.

STEP 1 · SETUP

A local model with Ollama

Terminal

```
# 1. Install Ollama: https://ollama.com/download
# 2. Pull the model used in class (2.2 GB)
ollama pull granite4.1:3b
# 3. Smoke test
ollama run granite4.1:3b "Say hello in five words"
# Ollama now serves an HTTP API on localhost:11434
curl http://localhost:11434/api/tags
```

granite4.1:3b | 2.2 GB | 128K context | tool calling | thinking mode

Why a local model?

- **Private and free:** prompts and data never leave the laptop.
- **Works offline:** no API keys, no rate limits, no bills.
- **Same model for all:** everyone gets comparable answers.
- **Trade-off:** a 3B model makes more mistakes, which is exactly why we validate.

JAVA IN ONE SLIDE

The Java you will see today

A trimmed-down Step2TypedAnswers.java

```
import dev.langchain4j.service.UserMessage;

public class Step2TypedAnswers {

    record Listing(int bedrooms, int rent) {}

    enum Decision { PUBLISH, REVIEW, REJECT }

    interface ListingAssistant {
        @UserMessage("Extract the details: {{notes}}")
        Listing extract(String notes);
    }

    public static void main(String[] args) {
        ChatModel model = OllamaChatModel.builder()
                .modelName("granite4.2:3b")
                .build();
    }
}
```

class and main

A file holds one public class. The program starts at main.

import

Uses a class from a library,

record

A small class that only holds data: listing.rent() reads a field.

enum

A fixed list of labels. The answer must be one of them.

interface

Methods without bodies. LangChain4j writes the bodies for us.

@Annotation, builder

@UserMessage is a label that LangChain4j reads. A builder sets one option per line, then .build().

STEP 1 · FIRST CALL

Hello, local model

ChatModel: text in, text out

The simplest way to use a model: send a String, get a String back.

The builder says where Ollama runs and which model to use.

▷ **Demo:** Step1FirstCall

Output of one real run

Enjoy bright, sunny living in a charming 2-bedroom apartment just minutes from UVic, perfect for students and professionals alike.

Just a String: we can't check it or rely on its shape. The next steps fix that.

Step1FirstCall.java, the whole file

```java
package realestate;

import dev.langchain4j.model.chat.ChatModel;
import dev.langchain4j.model.ollama.OllamaChatModel;

public class Step1FirstCall {

    public static void main(String[] args) {
        // Connect to the model that Ollama runs here
        ChatModel model = OllamaChatModel.builder()
                .baseUrl("http://localhost:11434")
                .modelName("granite4.2:3b")
                .build();

        // Text in, text out
        String question = "Write a one-sentence ad for "
                + "a sunny 2-bedroom apartment near UVic.";
        String answer = model.chat(question);
        System.out.println(answer);
    }
}
```

STEP 2 · TYPED ANSWERS

Answers as Java types

Step2TypedAnswers.java: the types

```
public class Step2TypedAnswers {
    record Listing(int bedrooms, int rent,
            boolean petsAllowed, String neighbourhood) {}
    enum Decision { PUBLISH, REVIEW, REJECT }
    interface ListingAssistant {
        @UserMessage("Extract the listing details from "
                + "these notes: {{notes}}")
        Listing extract(String notes);
        @UserMessage("Screen this rental listing. "
                + "PUBLISH: nothing unusual. "
                + "REVIEW: odd details that a person "
                + "should check. "
                + "REJECT: clear scam signs, like money "
                + "before any viewing. "
                + "Listing: {{listing}}")
        Decision screen(String listing);
    }
```

...and main

```
    public static void main(String[] args) {
        ChatModel model = OllamaChatModel.builder()
                .baseUrl("http://localhost:11434")
                .modelName("granite4.2:3b")
                .temperature(0.0)  // no randomness
                .build();
        ListingAssistant assistant = AiServices.create(
                ListingAssistant.class, model);
        String notes = "Bright 2 bed suite in Fernwood, "
                + "$2,350 a month, cats OK.";
        Listing listing = assistant.extract(notes);
        System.out.println(listing);
        System.out.println(listing.rent() + 75); // an int
        String scam = "Condo, $800. I'm abroad: wire the "
                + "deposit and I'll mail the keys.";
        Decision decision = assistant.screen(scam);
        System.out.println(decision);
    }
}
```

Output: `Listing[bedrooms=2, rent=2350, petsAllowed=true, neighbourhood=Fernwood]  2425  REJECT`

STEP 2 · TYPED ANSWERS

What LangChain4j does on each call

1 Fill in the template
{{notes}} becomes the owner's notes

2 Add format instructions
Built from the return type: Listing has 4 typed fields

3 Send it to the model
POST localhost:11434/api/chat, temperature 0

4 Parse the answer into the record
{"bedrooms": 2, "rent": 2350, ...} becomes Listing[bedrooms=2, rent=2350, ...]

⚠ Types are guaranteed; values are not. That is what step 3 is for.

What the model receives (seen with logRequests)

```
Extract the listing details from these notes:
Bright 2 bed suite in Fernwood, $2,350 a month,
cats OK.
You must answer strictly in the following JSON
format: {
"bedrooms": (type: integer),
"rent": (type: integer),
"petsAllowed": (type: boolean),
"neighbourhood": (type: string)
}
```

Yellow: added by LangChain4j, from the type Listing.

What it answers

```
{"bedrooms": 2, "rent": 2350,
 "petsAllowed": true, "neighbourhood": "Fernwood"}
```

For an enum, it adds: "You must answer strictly with one of these enums: PUBLISH, REVIEW, REJECT".

STEP 3 · CHECK AND REPAIR

Instruct, validate, repair, in plain Java

1 · INSTRUCT

Ask the model for a first answer: a rental ad.

```
writer.write(notes)
```

2 · VALIDATE

Plain Java checks the ad: at most 60 words, and the rent is there.

```
check(ad) → null or problem
```

3 · REPAIR

Send the ad back with the problem; the model fixes it. At most 3 tries.

```
writer.fix(ad, problem)
```

All checks pass: use the answer.

Still failing after 3 tries: throw an exception. Fail loudly.

Good to know: LangChain4j can run this loop for you (output guardrails). Writing it by hand first shows exactly what happens.

STEP 3 · CHECK AND REPAIR

Check and repair, in one class

The writer and the requirement

```
public class Step3CheckAndRepair {
    interface AdWriter {
        @UserMessage("Write a short rental ad from "
                + "these notes: {{notes}}")
        String write(String notes);
        @UserMessage("Rewrite this rental ad to fix one "
                + "problem: {{problem}} Answer with the "
                + "new ad only. Ad: {{ad}}")
        String fix(@V("ad") String ad,
                   @V("problem") String problem);
    }
    static String check(String ad) {
        int words = ad.split("\\s+").length;
        if (words > 60) {
            return "It has " + words + " words; use at "
                    + "most 60 words, in one paragraph.";
        }
        if (!ad.contains("$2,350")) {
            return "Say that the rent is $2,350 a month.";
        }
        return null;
    }
```

main (the model is built as in step 2)

```
        AdWriter writer =
                AiServices.create(AdWriter.class, model);
        String notes = "2 bed suite, Fernwood, "
                + "$2,350/month, cats OK, near bus 22, "
                + "shared yard";
        String ad = writer.write(notes);     // instruct
        String problem = check(ad);          // validate
        int tries = 1;
        while (problem != null && tries < 3) {
            System.out.println("Repairing: " + problem);
            ad = writer.fix(ad, problem);    // repair
            problem = check(ad);
            tries++;
        }
        if (problem != null) {               // fail loudly
            throw new IllegalStateException(
                    "Failed after 3 tries: " + problem);
        }
        System.out.println(ad);
```

STEP 3 · CHECK AND REPAIR

One real run: watch the repair

Step3CheckAndRepair, default temperature. Every run differs; this one needed one repair.

`user`	Write a short rental ad from these notes: 2 bed suite, Fernwood, $2,350/month, cats OK, near bus 22, shared yard	
`model`	**Spacious 2-Bed Suite in Fernwood - $2,350/month** Looking for a cozy home in Fernwood? We have the perfect place for you! This beautifully maintained 2-bedroom suite is ready … (97 words)	**fails**
`check()`	It has 97 words; use at most 60 words, in one paragraph.	**the problem**
`user`	Rewrite this rental ad to fix one problem: It has 97 words; use at most 60 words, in one paragraph. Answer with the new ad only. Ad: **Spacious 2-Bed Suite …	**the repair**
`model`	Looking for a cozy home in Fernwood? This beautifully maintained 2-bedroom suite is ready for your next chapter. Conveniently located near Bus Line 22, it features fully furnished living, welcomes cats, and offers a shared yard for relaxation. Enjoy the neighborhood at $2,350 per month. Visit us today for a tour! (51 words)	**passes**

⚠ **It passes both checks, yet it invents “fully furnished”.** Checks catch only what you test for (lab task 5).

STEP 4 · MAJORITY VOTE

Buy reliability with more calls

Check and retry

Step 3: ask again until the checks pass.

Majority vote

Ask n times and keep the most common answer. Good for labels.

Best of N

Get n answers, score each with code or a model, keep the best. Good for text.

temperature(0.8): answers may now differ. Five votes cost five calls.

Voting fixes random slips, not steady mistakes: “send a copy of your ID first” got PUBLISH 5 times out of 5.

Step4MajorityVote.java, inside main

```
String listing = "Studio near UVic, $1,100 a "
        + "month. Owner travelling; pay a deposit "
        + "to hold it.";
// Ask 5 times and count the votes for each decision
Map<Decision, Integer> votes = new HashMap<>();
for (int i = 0; i < 5; i++) {
    Decision d = screener.screen(listing);
    votes.put(d, votes.getOrDefault(d, 0) + 1);
}
System.out.println("Votes: " + votes);
// Keep the decision with the most votes
Decision winner = null;
for (Decision d : votes.keySet()) {
    if (winner == null
            || votes.get(d) > votes.get(winner)) {
        winner = d;
    }
}
System.out.println("Majority decision: " + winner);
```

One real run: `Votes: {REVIEW=5}  Majority decision: REVIEW`

LAB

Extend the listing platform

1	**Change the schema**	Step 2: add boolean parkingIncluded to Listing. Turn on logRequests and find the new field in the prompt.
2	**Add a requirement**	Step 3: also check that the ad says Fernwood. Run it until you see a repair; keep the output.
3	**Vote**	Step 4: vote on “Furnished studio, $1,050. Viewing after you send a copy of your ID.” Is the majority right? Why can’t voting fix it?

WRAP-UP

Takeaways

1. Treat LLM calls as **typed functions**: the return type is the contract.
2. Requirements are **code**: check, repair, and fail loudly when the tries run out.
3. Voting buys **reliability with more calls**, but can't fix a steady mistake.
4. Tools let the model use your data; keep the **control flow in Java** when you can.
5. Test the rules, **measure the model**, and log every call.

Resources

docs.langchain4j.dev: AI Services, tools, memory

ollama.com/library/granite4.2

Maven in 5 minutes (maven.apache.org)

Companion project: genai-java-lab

Questions?

UVIC